IEEE Access
Multidisciplinary · Rapid Review · Open Access Journal

A RISC-V Custom-Instruction Accelerator for Fixed-Point Min-Sum LDPC Decoding on FPGA

ANAS TAREK GEBREEL[1]

[1]*Faculty of Electrical Engineering, Universiti Teknologi Malaysia (UTM), Johor Bahru, Malaysia (e-mail: anastarek03@graduate.utm.my)*

Corresponding author: Anas Tarek Gebreel (e-mail: anastarek03@graduate.utm.my).

This work was carried out during industrial training at the VLSI and Embedded Computing Architecture Design (VeCAD) research laboratory, UTM, under the supervision of Dr. Jeevan.

ABSTRACT Low-Density Parity-Check (LDPC) codes provide near-capacity error correction and are used in modern standards such as 5G, Wi-Fi, satellite, and storage. Their iterative message-passing decoders are computationally intensive, which motivates dedicated hardware. This paper presents the design and hardware evaluation of a custom-instruction accelerator for the min-sum LDPC check-node update, integrated into an open-source RISC-V soft processor (NEORV32) on an Intel Cyclone IV E FPGA (Terasic DE2-115). A software min-sum decoder is first taken as a baseline and quantized from 32-bit floating point to 5- and 6-bit fixed point; Monte-Carlo simulation over an additive white Gaussian noise channel shows that 5-to-6-bit precision matches the floating-point bit-error rate with no measurable loss. The quantized check-node kernel, which computes the minimum and second-minimum magnitudes and the product of signs, is then implemented as a single RISC-V custom instruction and invoked from C. Measured on hardware, the accelerator reduces the cost of a check-node update from approximately 140 to 26 clock cycles, a speedup of about five times, while adding only 128 logic elements (roughly 0.1 percent of the device) and 27 registers. The complete processor-plus-accelerator system uses 4,082 logic elements (4 percent of the device) and meets timing at the 50 MHz operating clock, with a maximum clock frequency of 59.04 MHz. The results demonstrate that a compact fixed-point custom instruction can substantially accelerate LDPC decoding on a soft processor at negligible area cost and with no loss of coding performance.

INDEX TERMS Custom instruction, FPGA, fixed-point quantization, hardware acceleration, LDPC codes, min-sum decoding, RISC-V.

I. INTRODUCTION

Error-correcting codes are essential to reliable digital communication and storage, allowing a receiver to detect and repair the bit errors introduced by channel noise. Low-Density Parity-Check (LDPC) codes, first proposed by Gallager in 1962 [2], are defined by a sparse parity-check matrix and decoded by iterative message passing on a bipartite graph. After being largely dormant for three decades, they were rediscovered in the 1990s [4] and shown to operate close to the Shannon limit [1]. Today LDPC codes are part of many standards, including 5G New Radio, Wi-Fi, DVB-S2 satellite broadcasting, and solid-state storage.

The dominant practical decoding algorithm is the min-sum approximation to belief propagation [5], which replaces the exact but expensive check-node computation with a minimum-magnitude and sign-product operation. Because the check-node and variable-node updates are independent, LDPC decoding maps naturally onto parallel hardware, and dedicated decoders are widely used. However, on a general-purpose processor the message-passing loop is executed serially and is comparatively slow.

This paper investigates accelerating min-sum LDPC decoding on a soft RISC-V processor by adding the check-node update as a single custom instruction, and evaluates the resulting cost and performance on a low-cost FPGA. The contributions are: (i) a quantization study establishing the minimum arithmetic precision needed for a hardware min-sum decoder; (ii) the design of a fixed-point min-sum check-node kernel and its integration into the NEORV32 RISC-V custom-functions unit; and (iii) a hardware evaluation of decoding speedup, logic-resource cost, and maximum clock frequency on an Intel Cyclone IV E FPGA.

II. BACKGROUND

A. LDPC CODES AND THE TANNER GRAPH

An LDPC code is a linear block code specified by a sparse parity-check matrix H of size $m \times n$. A binary vector $\boldsymbol{c}$ is a valid codeword if and only if

$$H\mathbf{c} = \mathbf{0} \pmod 2 \qquad (1)$$

Each row of H is a parity-check equation, and the code is represented as a Tanner graph with n variable nodes (bits) and m check nodes (equations), an edge joining a variable to a check wherever H has a one.

B. MIN-SUM DECODING

Soft-decision decoding operates on log-likelihood ratios (LLRs). For a bit transmitted with binary phase-shift keying over an additive white Gaussian noise (AWGN) channel with noise standard deviation σ, the LLR of a received sample r is

$$L = \log\frac{P(b=0 \mid r)}{P(b=1 \mid r)} = \frac{2r}{\sigma^2} \qquad (2)$$

The check-node update sends each connected bit a message. The exact (sum-product) rule uses hyperbolic tangents, which are costly in hardware; the min-sum approximation replaces it by

$$C_{j\to i} = \left(\prod_{k\neq i} \operatorname{sign}(V_{k\to j})\right) \min_{k\neq i} |V_{k\to j}| \qquad (3)$$

Each variable node then sums its channel LLR and all incoming check messages, makes a tentative hard decision, and the syndrome $s = Hd$ is tested; decoding stops when the syndrome is zero or an iteration limit is reached.

III. METHODOLOGY

A. BASELINE DECODER AND QUANTIZATION

A regular rate-1/2 LDPC code of length $n = 128$ with variable-node degree three and check-node degree six was used for the software study. A min-sum decoder was implemented in C as the floating-point baseline. Fixed-point versions were produced with a uniform, saturating quantizer of Q bits, where each value is rounded

to a grid and clipped to $\pm(2^{Q-1}-1)$. For a target E_b/N_0 the channel noise is

$$\sigma = \sqrt{\frac{1}{2R \cdot \mathrm{SNR}}}, \quad \mathrm{SNR} = 10^{(E_b/N_0)/10} \tag{4}$$

with rate $R = 1/2$. Bit-error rate (BER) was measured by Monte-Carlo simulation, transmitting the all-zero codeword and averaging over 1.5×10^4 frames per operating point.

B. CUSTOM-INSTRUCTION ACCELERATOR

The target platform is the open-source NEORV32 RISC-V processor synthesized on a Terasic DE2-115 board (Intel Cyclone IV E, EP4CE115F29C7). NEORV32 provides a custom-functions unit (CFU) that adds user-defined RISC-V instructions accessible from C. The min-sum check-node kernel was implemented in this unit for a check of degree six with 5-bit messages: six signed messages are packed into a single 32-bit source operand, and the kernel computes, combinationally, the smallest magnitude m_1, the second-smallest m_2, the index of m_1, and the product of all signs. From these, the outgoing message for each edge is reconstructed as the product of the other signs times m_2 if that edge is the minimum index, or m_1 otherwise. The kernel returns its packed result in one custom instruction, replacing the software loop that scans the incoming messages. Correctness was verified against the C reference model.

IV. RESULTS

A. QUANTIZATION: CODING PERFORMANCE

Table 1 compares the BER of the 32-bit floating-point decoder with the 6-, 5-, and 4-bit fixed-point versions, and Fig. 1 plots the curves. The 6-bit, 5-bit, and floating-point curves are statistically identical, demonstrating that reducing the arithmetic precision to five or six bits incurs no measurable coding loss. At four bits a small penalty of roughly 0.2 dB appears, and at three bits the decoder degrades sharply. This fixes the datapath width for the hardware kernel at five to six bits.

TABLE 1. Bit-error rate: floating point versus fixed point (length-128, rate-1/2 LDPC, min-sum, AWGN).

E_b/N_0 (dB)	32-bit float	6-bit	5-bit	4-bit
3.0	4.99×10^{-3}	4.75×10^{-3}	4.83×10^{-3}	6.98×10^{-3}
3.5	1.24×10^{-3}	1.26×10^{-3}	1.35×10^{-3}	2.01×10^{-3}
4.0	3.33×10^{-4}	3.42×10^{-4}	3.43×10^{-4}	4.17×10^{-4}
4.5	2.54×10^{-5}	3.19×10^{-5}	3.06×10^{-5}	6.05×10^{-5}

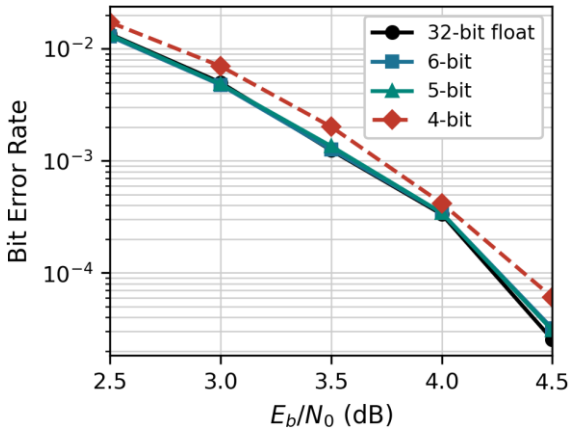


FIGURE 1. Bit-error rate versus E_b/N_0 for the floating-point decoder and the 6-, 5-, and 4-bit fixed-point versions. The 5- and 6-bit curves overlap the floating-point reference; 4-bit shows a small penalty.

B. HARDWARE ACCELERATION

Table 2 summarizes the measured hardware results. The custom instruction reduces a check-node update from approximately 140 to 26 clock cycles (Fig. 2), a speedup of about five times, while adding only 128 logic elements and 27 registers relative to the baseline processor. The complete processor-plus-accelerator system occupies 4,082 logic elements (4 percent of the device) and meets timing at the 50 MHz operating clock, with a reported maximum clock frequency of 59.04 MHz at the slow 85 °C corner.

TABLE 2. Measured hardware results on the Cyclone IV E (EP4CE115F29C7).

Metric	Result
Software check-node update	≈140 cycles
Accelerated (CFU) update	≈26 cycles
Speedup	≈5×
CFU logic elements	128 (≈0.1%)
CFU registers	27
System logic elements	4082 (4%)
System registers	2241
Maximum clock (Fmax)	59.04 MHz
Coding-performance loss	none (5-6 bit)

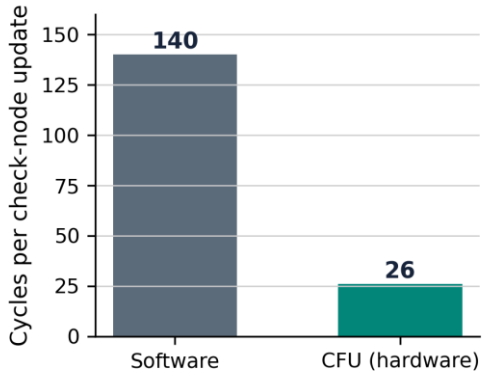


FIGURE 2. Measured clock cycles for one min-sum check-node update: software loop versus the custom-instruction (CFU) accelerator on the NEORV32 RISC-V core.

C. AREA OVERHEAD OF THE ACCELERATOR

To isolate the cost of the accelerator itself, the processor was synthesized twice: once with the CFU disabled and once with it enabled. Table 3 lists the two builds. Enabling the CFU adds only 128 logic elements and 27 registers, about 0.1 percent of the FPGA, so the speedup is obtained at negligible additional hardware.

TABLE 3. Resource use with and without the custom-instruction accelerator (Cyclone IV E).

Configuration	Logic elements	Registers
Processor only (CFU off)	3,954	2,214
Processor + CFU	4,082	2,241
Overhead (CFU)	+128	+27

D. COMPARISON

Two comparisons are drawn. First, against the 32-bit floating-point baseline in Table 1, the proposed 5-to-6-bit fixed-point decoder incurs no measurable coding loss; the bit-error rate of the accelerated decoder is therefore identical to that of the full-precision reference. Second, the required precision is consistent with published results: Zhao et al. [7] report that min-sum decoding attains near-ideal performance with only four to six quantization

bits, in close agreement with the 5-to-6-bit finding obtained here. The present work extends that observation by mapping the quantized check-node kernel to a single RISC-V custom instruction and measuring the resulting decode-time reduction and area cost directly on hardware.

V. DISCUSSION

The two results are complementary. The quantization study shows that the algorithm tolerates aggressive precision reduction, which is a precondition for efficient hardware; the hardware measurement then shows that, at that precision, the check-node kernel can be offloaded to a single custom instruction for a fivefold reduction in cycles at negligible area cost. The small footprint, about 0.1 percent of a low-cost device, indicates that many such units could be instantiated for a fully parallel decoder, and that the approach scales to the larger, structured (quasi-cyclic) codes used in standards. The exact speedup depends on the check-node degree and the surrounding loop overhead; a degree-six kernel was used here, and larger degrees would increase the per-instruction work offloaded and hence the speedup.

VI. CONCLUSION AND FUTURE WORK

This paper demonstrated a compact RISC-V custom-instruction accelerator for fixed-point min-sum LDPC decoding on a low-cost FPGA. A 5-to-6-bit fixed-point decoder was shown to match floating-point coding performance, and the quantized check-node kernel, integrated into the NEORV32 custom-functions unit, decoded each check about five times faster than software while adding only 128 logic elements and meeting timing at 50 MHz. Future work includes extending the accelerator to a standard quasi-cyclic LDPC code, adding normalized or offset min-sum to recover the small approximation loss, and a full energy-per-bit evaluation. More broadly, the same sparse-parity-check and message-passing framework now underpins quantum LDPC codes, where belief-propagation decoders are being adapted for fault-tolerant quantum memories [8]; the hardware techniques studied here are therefore relevant to both classical and emerging quantum error correction.

ACKNOWLEDGMENT

The author thanks Dr. Jeevan and the VeCAD research laboratory, UTM, for their guidance and support during the industrial training.

REFERENCES

[1] C. E. Shannon, “A mathematical theory of communication,” Bell Syst. Tech. J., vol. 27, pp. 379-423, 623-656, 1948.

[2] R. G. Gallager, “Low-density parity-check codes,” IRE Trans. Inf. Theory, vol. 8, no. 1, pp. 21-28, Jan. 1962.

[3] R. M. Tanner, “A recursive approach to low complexity codes,” IEEE Trans. Inf. Theory, vol. 27, no. 5, pp. 533-547, Sep. 1981.

[4] D. J. C. MacKay and R. M. Neal, “Near Shannon limit performance of low density parity check codes,” Electron. Lett., vol. 32, no. 18, pp. 1645-1646, 1996.

[5] M. P. C. Fossorier, M. Mihaljevic, and H. Imai, “Reduced complexity iterative decoding of low-density parity check codes based on belief propagation,” IEEE Trans. Commun., vol. 47, no. 5, pp. 673-680, May 1999.

[6] T. J. Richardson and R. L. Urbanke, “The capacity of low-density parity-check codes under message-passing decoding,” IEEE Trans. Inf. Theory, vol. 47, no. 2, pp. 599-618, Feb. 2001.

[7] J. Zhao, F. Zarkeshvari, and A. H. Banihashemi, “On implementation of min-sum algorithm and its modifications for decoding low-density parity-check (LDPC) codes,” IEEE Trans. Commun., vol. 53, no. 4, pp. 549-554, Apr. 2005.

[8] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, “High-threshold and low-overhead fault-tolerant quantum memory,” Nature, vol. 627, pp. 778-782, 2024.